

Curso – Técnico e Desenvolvimento de Sistemas

Professor- Hudson Neves

Alunos – Guilherme, Luiz Felipe, Gabriel Toledo, Cauê

RELATÓRIO DE CONCLUSÃO DE PROJETO

1. INTRODUÇÃO

O presente relatório tem por objetivo apresentar os resultados obtidos no desenvolvimento do projeto de conclusão, que consiste em um sistema web completo de agendamento online para barbearia. O sistema foi desenvolvido em formato full-stack, integrando uma API REST como camada de backend com uma interface de usuário moderna como camada de frontend.

O projeto simula um ambiente real de negócio, no qual clientes podem criar contas, autenticar-se, visualizar serviços e produtos disponíveis, e agendar horários diretamente pelo site, de forma prática e segura.

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver uma aplicação web full-stack funcional que contemple uma API RESTful robusta integrada a uma interface de usuário moderna, aplicando boas práticas de desenvolvimento de software, segurança e testes automatizados.

2.2 Objetivos Específicos

Implementar uma API REST com Node.js e Express.js para gerenciamento de usuários, serviços, agendamentos e produtos;

Desenvolver autenticação segura com criptografia de senhas e tokens JWT;

Criar uma interface de usuário responsiva com React.js;

Aplicar testes automatizados para validar as funcionalidades críticas do sistema;

Organizar o código seguindo o padrão de arquitetura MVC (Model-View-Controller).

3. DESCRIÇÃO DO SISTEMA

O sistema foi desenvolvido para atender às necessidades de uma barbearia moderna, permitindo que clientes realizem agendamentos de forma online, sem necessidade de ligações telefônicas. O sistema conta com um painel administrativo e uma área do cliente, oferecendo uma experiência completa para ambos os perfis de usuário.

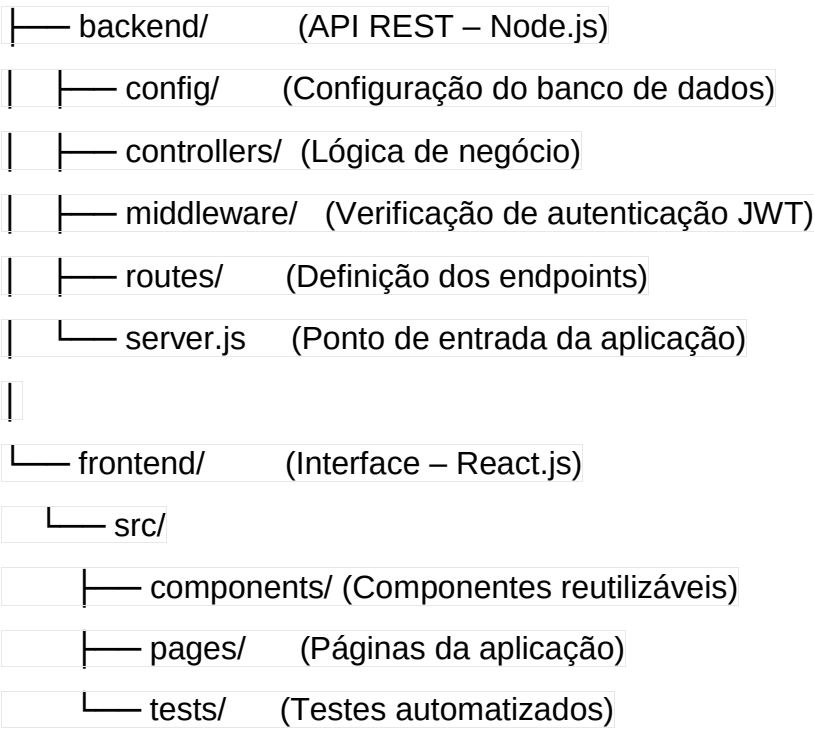
3.1 Funcionalidades Implementadas

Funcionalidade	Descrição	Status
Cadastro de Usuário	Criação de conta com nome, e-mail e senha criptografada	✓ Concluído
Login/Autenticação	Autenticação com e-mail e senha, geração de token JWT	✓ Concluído
Controle de Acesso	Proteção de rotas via middleware com verificação de token	✓ Concluído
Listagem de Serviços	Consulta de serviços disponíveis com categoria e preço	✓ Concluído
Listagem de Produtos	Consulta de produtos disponíveis na barbearia	✓ Concluído
Agendamento Online	Criação de agendamento com verificação de conflito de horário	✓ Concluído
Histórico de Agendamentos	Visualização dos agendamentos do usuário logado	✓ Concluído
Dashboard Administrativo	Painel com resumo financeiro e gestão de agendamentos	✓ Concluído
Testes Automatizados	Suite de testes para autenticação e interface do usuário	✓ Concluído

4. ARQUITETURA DO SISTEMA

O projeto foi estruturado seguindo a arquitetura **MVC (Model-View-Controller)** e organizado em dois módulos principais:

barbearia/



5. API REST – ENDPOINTS IMPLEMENTADOS

A API foi desenvolvida seguindo os princípios REST, utilizando os métodos HTTP adequados para cada operação. Abaixo estão todos os endpoints disponíveis:

5.1 Autenticação (/api/auth)

Método	Endpoint	Descrição	Autenticação
POST	Api auth register	Cadastra novo usuário	Não
POST	api auth login	Realiza login e retorna token JWT	Não

5.2 Serviços (api services)

Método	Endpoint	Descrição	Autenticação
GET	Ap/ services	Lista todos os serviços disponíveis	Não

5.3 Agendamentos (/api/appointments)

Método	Endpoint	Descrição	Autenticação
GET	Api/ appointments	Lista agendamentos do usuário logado	Sim (JWT)
POST	Api/ appointments	Cria novo agendamento	Sim (JWT)

5.4 Produtos /api produtos)

Método	Endpoint	Descrição	Autenticação
GET	api/produtos	Lista todos os produtos disponíveis	Não

6. TECNOLOGIAS UTILIZADAS

6.1 Backend

Tecnologia	Versão	Finalidade
Node.js	LTS	Ambiente de execução JavaScript no servidor
Express.js	5.x	Framework para criação da API REST
JSON Web Token (JWT)	9.x	Autenticação stateless e controle de sessão
bcryptjs	3.x	Criptografia de senhas antes do armazenamento
SQLite / MySQL2	Ambos	Banco de dados relacional
dotenv	17.x	Gerenciamento de variáveis de ambiente
CORS	2.x	Política de compartilhamento entre origens
Nodemon	3.x	Reinicialização automática em desenvolvimento

6.2 Frontend

Tecnologia	Versão	Finalidade
React.js	19.x	Biblioteca para construção da interface de usuário
Vite	8.x	Ferramenta de build e servidor de desenvolvimento
React Router DOM	7.x	Gerenciamento de rotas client-side
Axios	1.x	Cliente HTTP para consumo da API
Lucide React	0.5x	Ícones modernos e acessíveis

6.3 Testes

Tecnologia	Versão	Finalidade
Vitest	4.x	Framework de testes unitários compatível com Vite
Testing Library (React)	16.x	Testes de componentes React
@testing-library/user-event	14.x	Simulação de interações do usuário
@testing-library/jest-dom	6.x	Matchers adicionais para o DOM
jsdom	29.x	Ambiente DOM simulado para execução dos testes

7. SEGURANÇA IMPLEMENTADA

A segurança foi uma prioridade no desenvolvimento do projeto. As seguintes medidas foram adotadas:

Criptografia de Senhas: Todas as senhas são armazenadas com hash utilizando bcryptjs com fator de custo 10, impedindo que senhas em texto puro sejam salvas no banco de dados;

Autenticação com JWT: Após o login, o servidor emite um Token JWT com prazo de expiração de 1 dia. Esse token é enviado no cabeçalho Authorization em todas as requisições protegidas;

Middleware de Autorização: A rota de agendamentos é protegida por um middleware que verifica e valida o token antes de processar qualquer requisição;

Controle de Conflito de Horários: O sistema verifica automaticamente se já existe um agendamento para o horário solicitado, impedindo sobreposições;

Variáveis de Ambiente: Dados sensíveis como a chave secreta do JWT e credenciais do banco de dados são armazenados em arquivo

.env, nunca expostos no código-fonte.

8. TESTES AUTOMATIZADOS

Foram desenvolvidas duas suites de testes automatizados para validar as funcionalidades críticas do sistema:

8.1 Resultados dos Testes

Suite de Teste	Caso de Teste	Resultado
Autenticação e Segurança	Renderização correta do formulário de login	✓ Passou
Autenticação e Segurança	Login com credenciais válidas e persistência do token JWT	✓ Passou
Autenticação e Segurança	Exibição de erro com credenciais inválidas	✓ Passou
Autenticação e Segurança	Preenchimento do formulário de cadastro (Register)	✓ Passou
Autenticação e Segurança	Limpeza de sessão no logout (remoção do token)	✓ Passou
Interface do Usuário	Indicadores financeiros e status no Dashboard administrativo	✓ Passou
Interface do Usuário	Renderização de botões e chamadas para ação na Home	✓ Passou
Interface do Usuário	Estrutura semântica adequada da página inicial	✓ Passou

Total de Testes: 8 | Aprovados: 8 | Reprovados: 0

8.2 Ferramentas e Abordagem

Os testes foram executados utilizando o Vitest como test runner integrado ao Vite. Foram utilizados mocks do Axios (`vi.mock('axios')`) para simular as respostas da API sem dependência da rede, garantindo testes rápidos e determinísticos. O `@testing-library/react` foi utilizado para renderizar componentes em um ambiente DOM simulado (`jsdom`).

9. CONSIDERAÇÕES FINAIS

O projeto foi concluído com êxito, atingindo todos os objetivos estabelecidos inicialmente. A construção de uma API REST do zero permitiu a compreensão aprofundada dos princípios de desenvolvimento web moderno, bem como das boas práticas de segurança e organização de código.

A separação clara entre backend e frontend — comunicados exclusivamente via API REST — evidencia uma arquitetura desacoplada e escalável, capaz de suportar futuras evoluções, como a integração com aplicativos mobile ou a migração para um banco de dados na nuvem.

A implementação dos testes automatizados comprova a estabilidade das funcionalidades mais críticas do sistema, especialmente o fluxo de autenticação, que é a base de segurança de toda a aplicação.

O projeto representa a consolidação dos conhecimentos adquiridos ao longo do curso, integrando disciplinas como Programação Web, Banco de Dados, Segurança da Informação e Engenharia de Software em uma solução funcional e aplicável ao mercado.

10. REFERÊNCIAS

MDN Web Docs. *HTTP Methods*. Disponível

em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Methods>

Express.js. *Documentação Oficial*. Disponível em: <https://expressjs.com/pt-br/>

React. *Documentação Oficial*. Disponível em: <https://react.dev/>

JSON Web Tokens. *Documentação Oficial*. Disponível

em: <https://jwt.io/introduction>

Vitest. *Documentação Oficial*. Disponível em: <https://vitest.dev/>

Testing Library. *Documentação Oficial*. Disponível em: <https://testing-library.com/>
